Pub-Sub Architecture

Objective

By reading this Pub/Sub Architecture documentation, readers will:

- Understand the core architectural principles of asynchronous messaging and event-driven systems.
- Master fundamental components including Publishers, Subscribers, Topics, Message Brokers, and Subscriptions.
- Learn to design robust, decoupled, and scalable microservice architectures using Pub/Sub patterns.
- Understand delivery semantics (At-Most-Once, At-Least-Once, Exactly-Once) and effective routing strategies.
- Gain insights into comparative tool decision-making (Kafka, RabbitMQ, Redis, Cloud-native services) and industry best practices for schema governance and security.

Outline

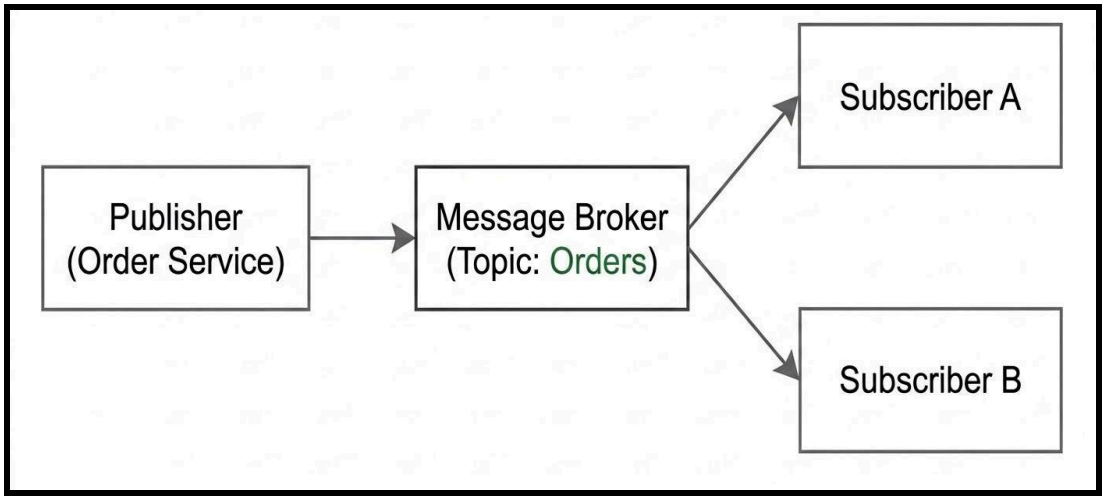
- Module 1: Introduction & Core Philosophy
- Module 2: Key Building Blocks & Vocabulary
- Module 3: Delivery Models & Message Filtering
- Module 4: Popular Tech Stack & Tool Comparisons
- Module 5: Real-World Use Cases & System Architecture Examples
- Module 6: Engineering Trade-offs & Common Pitfalls
- Module 7: Team Best Practices & Guidelines
- Summary

Introduction

What is Pub/Sub?

Publish-Subscribe (Pub/Sub) is an asynchronous messaging pattern designed to facilitate communication between decoupled software systems.

Instead of sending messages directly to specific receivers, sender applications - known as **Publishers**, broadcast messages to a central event bus or broker without knowing who (if anyone) will receive them. Receiver applications—known as **Subscribers**—express interest in specific categories or topics of messages and passively receive them as they arrive.



Overview of Asynchronous Messaging Patterns

In modern distributed systems and microservice architectures, asynchronous messaging enables services to execute tasks without waiting for an immediate response. Pub/Sub is one of several messaging patterns:

- **Point-to-Point (Queue):** A message is sent by a producer to a queue and processed by exactly *one* consumer. Once processed, the message is removed from the queue.
- **Publish-Subscribe (Topics):** A message is published to a topic and broadcast to *all* active subscribers registered to that topic (or filtered based on criteria).
- **Event Streaming:** Messages are recorded sequentially as an append-only log. Subscribers can read from any point in the stream and re-read historical data (e.g., Apache Kafka).

Contrast: Direct/Synchronous vs. Indirect/Asynchronous

Understanding when to use synchronous versus asynchronous patterns is fundamental to good architecture design.

Dimension	**Direct / Synchronous (Request-Response)**	**Indirect / Asynchronous (Event-Driven)**
Communication Style	Blocking (Client waits for server response)	Non-blocking (Publisher fires and forgets)

Protocols	REST, gRPC, HTTP/HTTPS	AMQP, MQTT, Kafka Protocol, WebSockets
Coupling	High: Client must know server endpoint and availability	Low: Neither party needs awareness of the other
Temporal Dependencies	Both services must be online simultaneously	Sender and receiver operate independently in time
Failure Handling	Cascading failures if downstream service is down	Messages are buffered until consumers recover
Best Used For	Real-time queries, transactional read/write ops	System notifications, background processing, logging

Key Takeaway: Use synchronous request-response when you need immediate feedback (e.g., authenticating a user password). Use Pub/Sub when an action triggers downstream effects that don't need to block the primary workflow (e.g., sending a welcome email after registration).

Why Use Pub/Sub?

1. Decoupling

Producers and consumers operate in total isolation:

- **Spatial Decoupling:** Publishers and subscribers do not need IP addresses, hostnames, or direct network access to each other; it's only needed to access message brokers.
- **Temporal Decoupling:** Publishers and subscribers do not need to be online at the same time. If a subscriber goes down, the broker holds the messages until it recovers.
- **Implementation Decoupling:** Services can be written in entirely different languages, frameworks, or tech stacks as long as they can serialize/deserialize the shared message format (e.g., JSON, Protobuf).

2. Scalability

Pub/Sub enables seamless horizontal scaling across your architecture:

- **Elastic Consumer Pools:** If message velocity spikes, you can spin up additional instances of a subscriber service to pull from the same topic or subscription queue without altering the publisher code or broker setup.
- **Fan-Out Architecture:** A single event can trigger multiple distinct downstream workflows simultaneously (e.g., an OrderPlaced event can trigger the Inventory, Payment, and Shipping services in parallel).

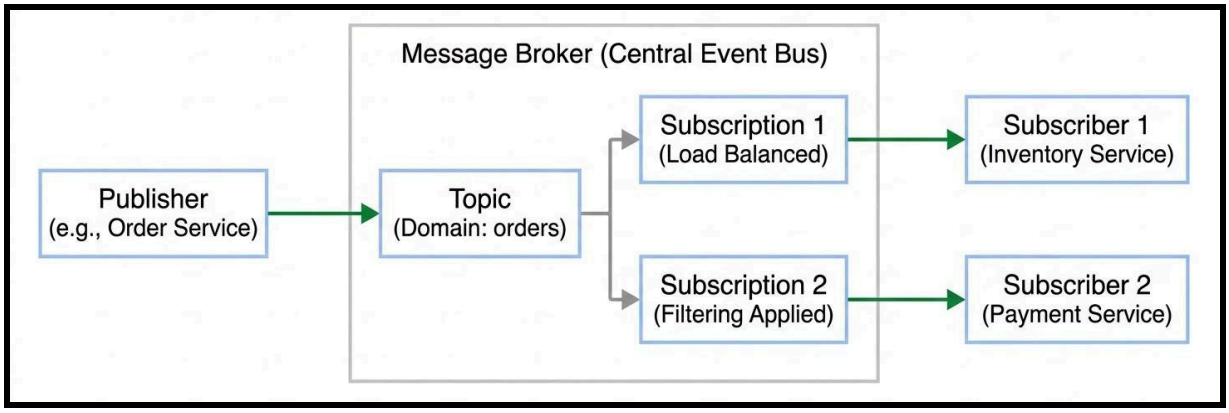
3. Resilience

By decoupling production from consumption, Pub/Sub acts as a safety buffer for system stability:

- **Traffic Spike Buffering (Load Leveling):** If a marketing campaign causes a 10x spike in traffic, the message broker absorbs the influx. Downstream services can process messages at their maximum sustainable rate rather than being crushed by request volume.
- **Fault Isolation:** If a downstream service crashes or undergoes maintenance, messages accumulate in the broker queue without causing errors for the upstream publisher or impacting end-user experience.

Key Building Blocks & Vocabulary

To design and build event-driven systems effectively, it is essential to establish a common language around the core components of the Pub/Sub architecture.



1. Publishers (Producers)

Publishers are applications, microservices, or components that generate data and emit events to the messaging system.

- **Role:** They detect system state changes (e.g., a user signing up, a payment completing) and construct a message to notify the rest of the ecosystem.

- **Key Characteristic:** Publishers are completely unaware of who reads their messages or how many consumers exist. They simply publish to a designated topic and return to their primary execution flow ("fire-and-forget").

2. Subscribers (Consumers)

Subscribers are applications or background workers that register interest in specific events and execute business logic upon receiving them.

- **Role:** They ingest incoming messages from the broker and perform downstream actions (e.g., updating a database, sending a push notification, triggering a data pipeline).
- **Key Characteristic:** Subscribers are completely decoupled from publishers. They process messages asynchronously and can scale up or down independently depending on throughput demands.

3. Topics / Channels

A **Topic** (sometimes called a Channel) is a named, logical routing path or category created within the message broker where related messages are published.

- **Role:** Topics act as organizational boundaries. Instead of sending messages to specific destinations, publishers send messages to a specific topic (e.g., user.created, payments.processed, sensor.telemetry).
- **Categorization Patterns:**
    - **Domain-based:** Grouping by business entity (e.g., orders, inventory).
    - **Action-based:** Grouping by event state change (e.g., order-shipped, user-deleted).

4. Message Broker / Event Bus

The **Message Broker** is the central infrastructure that orchestrates communication between publishers and subscribers.

- **Role:** It acts as the traffic controller—receiving published messages, validating access permissions, persisting data (if configured), and efficiently routing messages to the appropriate subscribers.
- **Core Responsibilities:**
    - Message buffering and queuing during high load.
    - Ensuring message persistence and delivery guarantees.
    - Managing subscriber connections and consumer group offsets.
- **Examples:** Apache Kafka, RabbitMQ, Google Cloud Pub/Sub, AWS SNS/SQS, Redis.

5. Subscriptions

A **Subscription** is the logical mapping or configuration rule that connects a subscriber to one or more topics.

- **Role:** It tells the message broker *how* and *where* to deliver messages from a topic to a consumer.

- **Types of Subscriptions:**
    - **Push Subscriptions:** The message broker initiates an HTTP request or Webhook call to deliver the message directly to the consumer endpoint.
    - **Pull Subscriptions:** The consumer regularly polls the message broker to request available messages.
    - **Shared / Group Subscriptions:** Multiple instances of a single consumer service share a single subscription to load-balance message handling across instances.

6. Payload

The **Payload** is the actual data payload carried within the envelope of a message. It represents the state change or event facts being communicated.

- **Role:** It contains the state or context needed for subscribers to perform their work.
- **Common Serialization Formats:**

Format	Description	Best Used For
JSON	Human-readable, flexible, dynamic schema format.	Web applications, rapid prototyping, heterogeneous systems.
Protocol Buffers (Protobuf)	Compact binary format with strict schema definitions. High performance.	Microservice architectures, gRPC systems, high-throughput pipelines.
Apache Avro	Binary format with built-in schema evolution support via schema registries.	Data engineering pipelines, event streaming (Kafka).

Example JSON Payload (order.created):

```
{
  "eventId": "evt_987654321",
  "eventType": "order.created",
  "timestamp": "2026-08-03T14:30:00Z",
  "data": {
    "orderId": "ord_102938",
```

```
    "customerId": "cust_554433",
    "totalAmount": 149.99,
    "currency": "USD"
  }
}
```

Delivery Models & Message Filtering

When building a Pub/Sub system, choosing the right delivery semantics and routing mechanisms determines your system's reliability, throughput, and complexity.

1. Delivery Strategies

Message delivery guarantees define how the message broker handles network failures, application crashes, and acknowledgments (ACKs).

Delivery Strategy	Method	Outcome
At-Most-Once	Fire & Forget	Might be lost
At-Least-Once	Retry until ACK	Guaranteed, but may cause duplicates
Exactly-Once	Transaction / Dedupe	Delivered once & only once (Costly)

A. At-Most-Once

The broker sends the message once and immediately assumes success without waiting for confirmation or acknowledgment from the consumer.

- **Behavior:** "Fire and forget." If a network blip occurs or the consumer service crashes mid-processing, the message is permanently lost.
- **Trade-offs:**
    - **Pros:** Lowest latency, highest throughput, minimal system complexity.
    - **Cons:** Zero delivery guarantees.
- **Ideal Use Cases:** High-frequency metrics, IoT telemetry, log streaming, live video streaming, or non-critical analytical data.

B. At-Least-Once

The broker guarantees that every message will reach the consumer at least once. It will repeatedly retry delivering the message until the consumer sends back an explicit **Acknowledgment (ACK)**.

- **Behavior:** If an ACK is lost due to a timeout or connection failure, the broker re-sends the message. As a result, the consumer may receive the same message multiple times.
- **Trade-offs:**
    - **Pros:** Guaranteed message delivery—no dropped data under normal network partitions.
    - **Cons:** Requires **Idempotent Consumers** (logic designed to produce the same result regardless of how many times the same message is processed).
- **Ideal Use Cases:** Financial processing, order processing, transactional updates—any scenario where missing data is unacceptable.

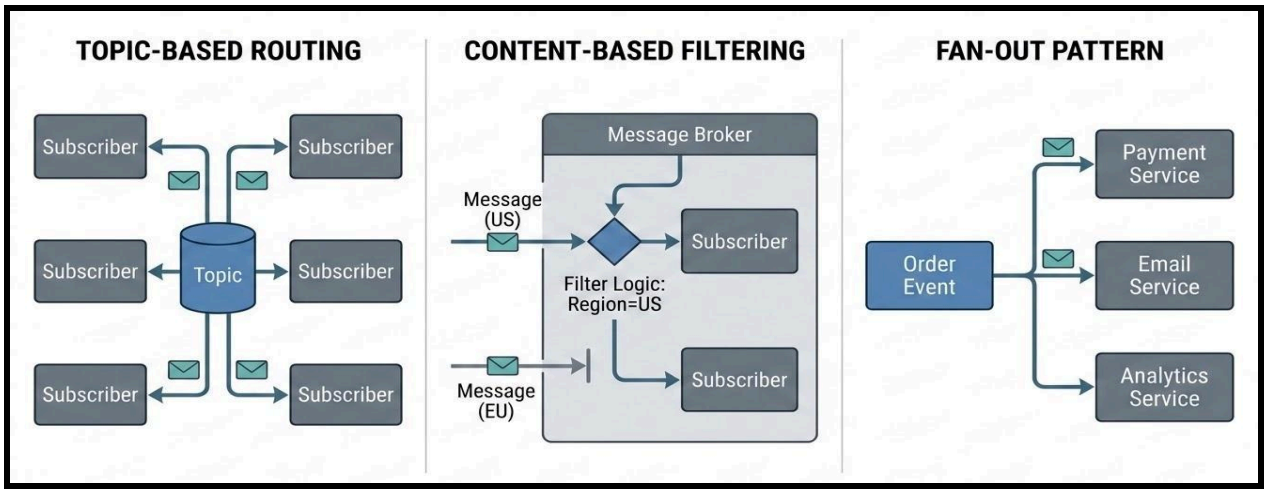
C. Exactly-Once

The broker and consumer work together using distributed transactions, unique message IDs, and deduplication mechanisms to ensure every message is processed **once and only once**.

- **Behavior:** Eliminates both data loss and duplicate processing at the framework level.
- **Trade-offs:**
    - **Pros:** Maximum reliability and simplified application-level logic.
    - **Cons:** Significant performance overhead, higher end-to-end latency, and complex broker configuration.
- **Ideal Use Cases:** Payment gateways, billing operations, inventory deduction logic where duplicate execution causes immediate state corruption.

2. Routing & Filtering Patterns

Routing mechanisms determine how a message moves from a broker to specific subscribers.



A. Topic-Based Routing

Subscribers register interest in a static topic name or pattern, receiving **all** messages published to that channel.

- **How It Works:** The broker evaluates message metadata against explicit topic names (e.g., `orders.created`) or wildcard patterns (e.g., `orders.*`).
- **Example:** Subscribing to `logs.error` delivers every error log in the system to the alerting service.

B. Content-Based Filtering

The message broker inspects the attributes, metadata headers, or payload content of a message to dynamically determine which subscribers should receive it.

- **How It Works:** Subscribers define rule-based predicates (e.g., `WHERE metadata.region == 'US' AND payload.amount > 1000`). The broker evaluates each message against these expressions before dispatching.
- **Advantage:** Subscribers avoid receiving unnecessary messages over the network, saving compute and bandwidth without creating dozens of granular topics.

C. Fan-Out Pattern

An architecture pattern where a single event published to a topic is simultaneously duplicated and distributed to **multiple distinct subscribers** executing independent tasks.

- **How It Works:** A single publisher emits an event once. The broker routes identical copies of the message to all subscribed queues in parallel.
- **Example (E-Commerce Workflow):**
    - An `OrderPlaced` event is published once by the Checkout Service.
    - The Fan-Out pattern broadcasts this single event simultaneously to:
        1. **Inventory Service:** Reserve stock.
        2. **Payment Service:** Charge the customer's card.
        3. **Notification Service:** Send an order confirmation email.
        4. **Fraud Service:** Run risk evaluation algorithms.

Popular Tech Stack & Tool Comparisons

Selecting the right messaging technology depends on your system's data volume, latency tolerance, routing complexity, and operational capacity.

1. Apache Kafka

- **How It Works:** A distributed append-only log where messages are written sequentially to disk and partitioned across a cluster.
- **Core Strengths:** Extremely high throughput (millions of msgs/sec), long-term message retention, horizontal scalability, and message replay capabilities.
- **Ideal Use Case:** Event-driven architectures, real-time streaming analytics, log aggregation, and audit/event-sourcing pipelines.

2. RabbitMQ

- **How It Works:** A traditional queue-based message broker implementing the AMQP protocol using exchanges, bindings, and queues to route messages.
- **Core Strengths:** Flexible and complex routing logic (direct, topic, fanout, headers), fine-grained priority queues, delayed retries, and high reliability.
- **Ideal Use Case:** Microservice background job processing, task queues, transactional messaging, and applications requiring complex routing.

3. Redis Pub/Sub

- **How It Works:** An in-memory, push-based broadcasting mechanism baked directly into Redis.
- **Core Strengths:** Ultra-low latency (sub-millisecond execution), simple setup with zero extra dependencies, and extremely lightweight footprint.
- **Ideal Use Case:** Ephemeral notifications, live chat applications, real-time dashboard updates, and short-lived state signaling.

4. Cloud Native Services (GCP Pub/Sub, AWS SNS/SQS, Azure Service Bus)

- **How It Works:** Fully managed, serverless messaging infrastructure hosted and scaled automatically by cloud providers.
- **Core Strengths:** Zero server maintenance, automatic elastic auto-scaling, pay-per-use pricing, and seamless security integration with cloud IAM systems.
- **Ideal Use Case:** Cloud-native microservices, serverless workloads (AWS Lambda / Cloud Functions), and cross-cloud event routing.

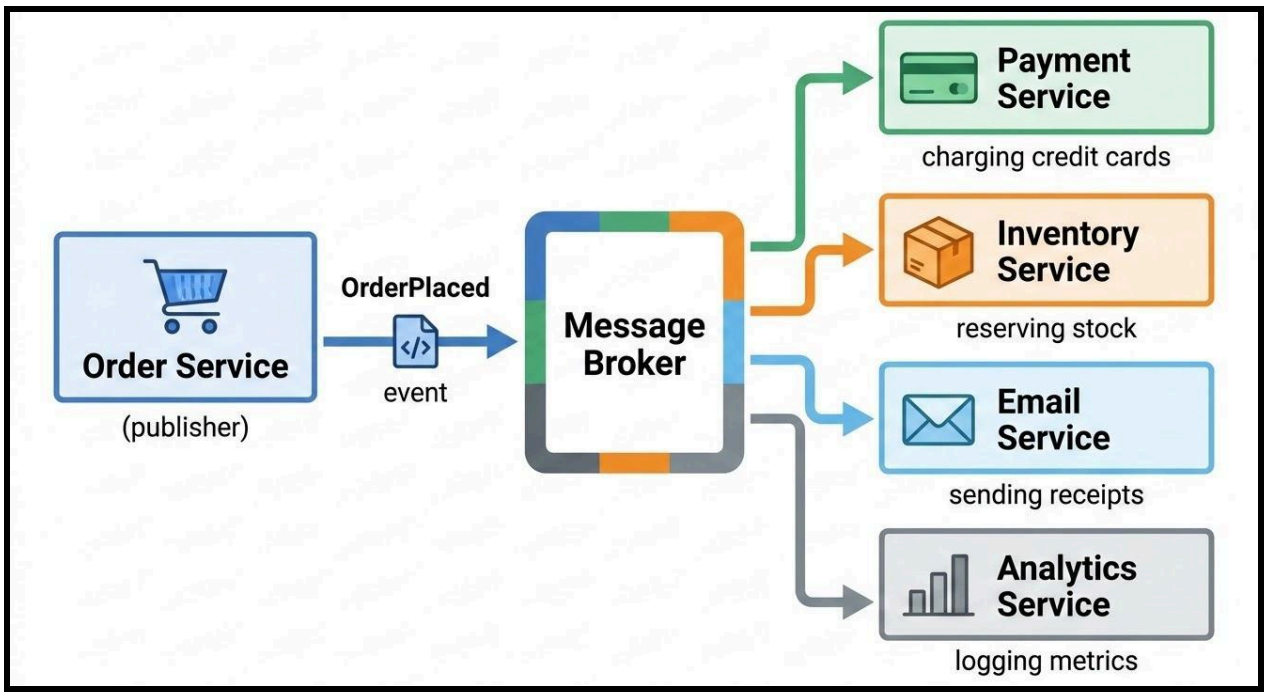
Real-World Use Cases & System Architecture Examples

To understand the real-world value of Pub/Sub, consider how modern event-driven architectures solve high-throughput and decoupled system requirements.

1. E-Commerce Checkout Flow (Parallel Task Processing)

In a traditional synchronous system, an **Order Service** calls the Payment, Inventory, Email, and Analytics services one after another via HTTP/REST. If one service fails or runs slowly, the entire checkout process stalls or crashes for the user.

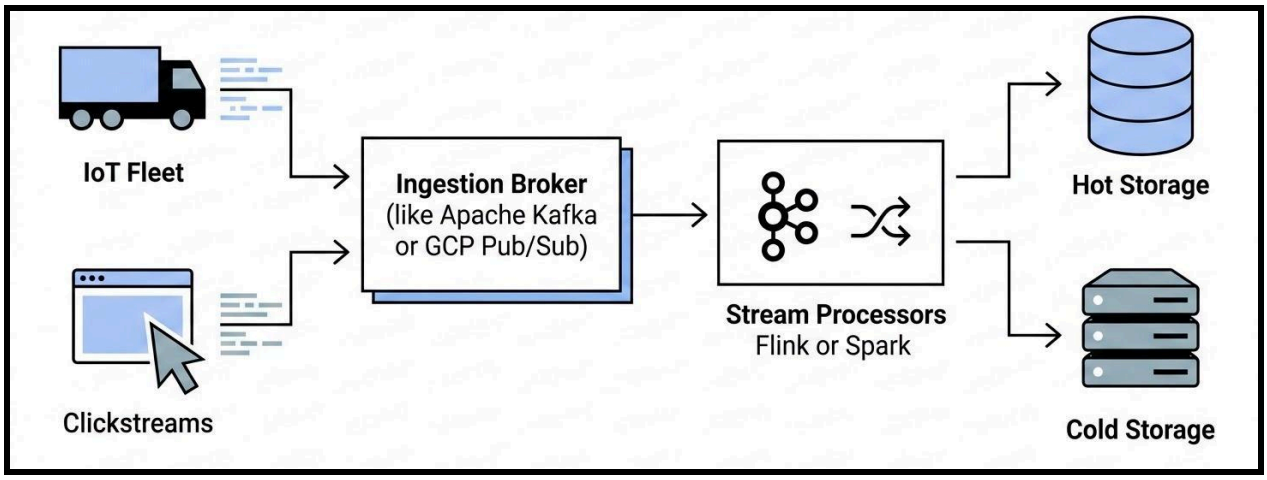
With Pub/Sub, the **Order Service** processes the request locally, publishes a single `OrderPlaced` event to a message broker, and immediately returns a success response to the customer.



- **Workflow Step-by-Step:**
    1. **Publisher:** The **Order Service** verifies order details and publishes an `OrderPlaced` event payload to the `orders` topic.
    2. **Broker:** The broker fans out the message to all registered subscribers in parallel.
    3. **Subscribers:**
        - **Payment Service:** Pulls the message and charges the user's payment method.
        - **Inventory Service:** Decrements stock counts in the warehouse database.
        - **Notification Service:** Constructs and queues an email receipt to the customer.
        - **Analytics Service:** Aggregates order data for real-time sales dashboards.
- **Key Advantage:** If the Email Service goes down due to an outage, the customer's purchase still completes without delay. The Email Service simply processes its pending queued messages once it recovers.

2. Real-Time Data Streaming (High-Volume Ingestion)

When a system demands scale to thousands or millions of events per second, Pub/Sub acts as an elastic buffer between data creators and data processors.



- **IoT Telemetry & Fleet Tracking:**
  - Thousands of connected devices (e.g., smart vehicles, medical sensors) emit continuous state updates (location, battery, temperature) every second.
  - Devices publish directly to cloud message brokers (e.g., GCP Pub/Sub or AWS IoT Core) via lightweight protocols like MQTT.
  - Subscribers process telemetry in real-time to alert operators of immediate failures (e.g., engine overheating).
- **User Clickstream Analytics:**
  - Web and mobile applications publish every user interaction (clicks, page views, search queries) to an event stream like Apache Kafka.
  - Real-time stream processors (e.g., Apache Flink, Spark Streaming) consume the data to update live recommendation engines, track ad conversions, and detect fraudulent user behavior instantly.
- **Key Advantage:** The ingestion layer handles massive traffic bursts without overloading long-term storage databases or analytics pipelines.

Pub/Sub Engineering Trade-offs & Common Pitfalls

While Publish/Subscribe (Pub/Sub) architectures solve major horizontal scalability and decoupling challenges, they introduce unique engineering trade-offs around data consistency, failure recovery, schema governance, and operational observability.

1. Message Ordering & Partitioning

In distributed systems, preserving strict message order across network partitions and horizontally scaled consumers is inherently difficult.

> **The Challenge:** Network delays, retry logic, and parallel processing can cause events to arrive out of order (e.g., an OrderCancelled event arrives before an OrderCreated event).

Global Ordering vs. Partitioning Keys

- **Global Ordering:** Forces all messages through a single broker queue or partition. While it guarantees strict sequence, it creates a massive performance bottleneck and eliminates parallel scaling capabilities.
- **Partition Keys (Recommended):** Messages are routed to specific partitions based on a consistent hash key (e.g., userId or orderId). Messages with the same key are guaranteed to be processed in sequence by the same consumer instance, while different keys scale across multiple consumers in parallel.

2. Idempotency & Delivery Guarantees

Due to the constraints of the CAP theorem, distributed message brokers default to *at-least-once* delivery semantics, making duplicate messages inevitable.

> **The Challenge:** Duplicate deliveries resulting from network timeouts, broker re-elections, or consumer crashes can corrupt system state (e.g., charging a credit card twice for a single order).

Delivery Semantics Spectrum

Semantic	Behavior	Use Case & Trade-offs
At-Most-Once	Messages delivered 0 or 1 time. Drops on failure.	High speed, zero duplicates; allows message loss (best for telemetry/metrics).
At-Least-Once	Messages delivered 1 or more times. Never drops.	Guarantees delivery, but introduces duplicates. Requires idempotent consumers.
Exactly-Once	Messages processed precisely once end-to-end.	Achieved via transactional coordinators or read-process-write loops; high performance overhead.

Strategies for Idempotent Consumers

- **Unique Message Deduplication IDs:** Client-generated idempotency keys (UUIDs) are passed in event metadata. Consumers track processed IDs in a high-speed cache (e.g., Redis) or database unique constraint to drop duplicates immediately.

- **Stateful Guarding (Idempotent DB Writes):** Construct operations so multiple executions produce identical state (e.g., using SET status = 'PAID' instead of INCREMENT balance, or applying conditional version checks).

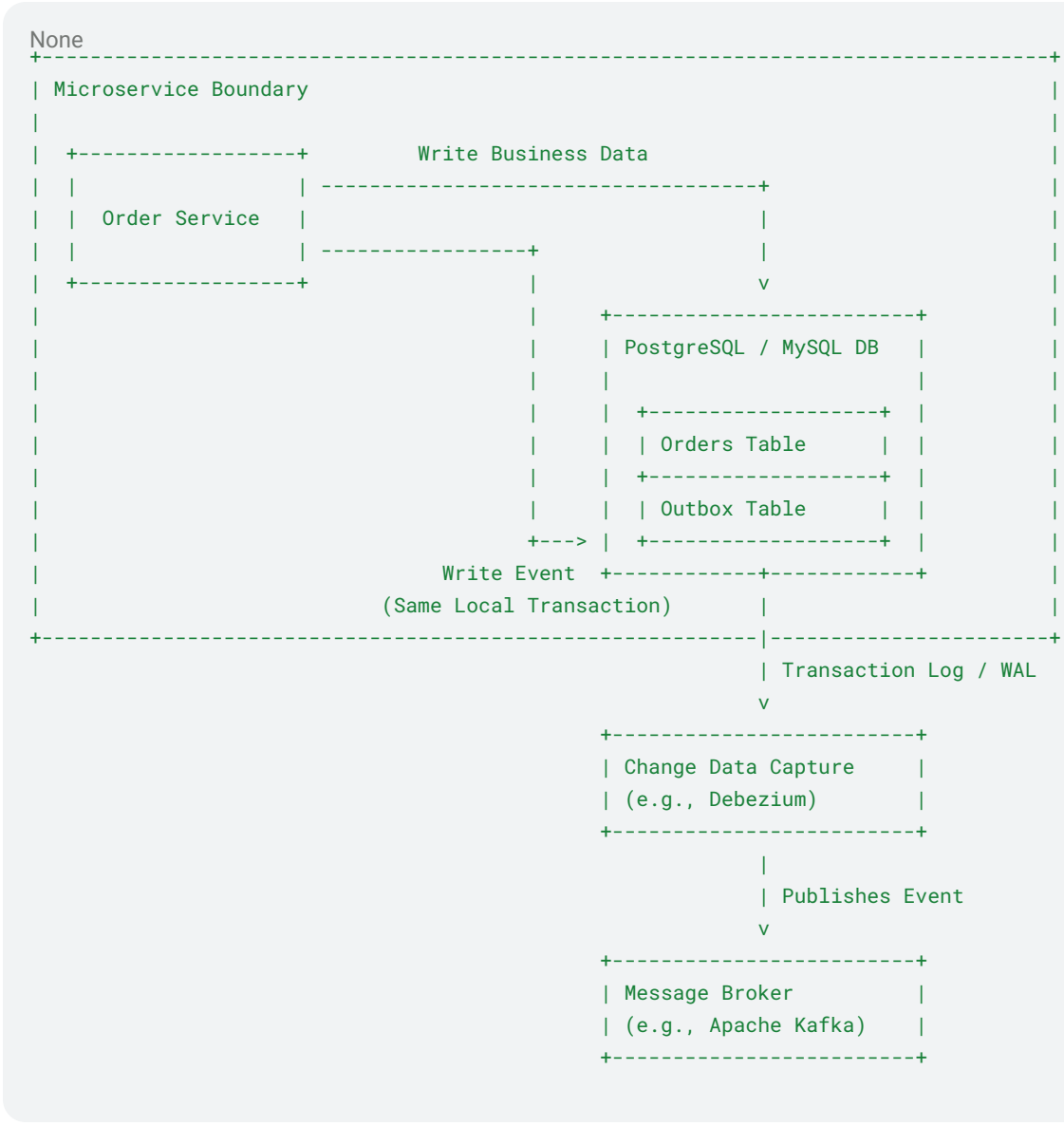
3. The Dual-Write Problem & Transactional Outbox

Publishing events alongside database writes creates an atomicity challenge across two distinct infrastructure components.

- **The Challenge:** If a service writes to a database and then attempts to publish an event directly to the broker, a failure between these two steps leaves the system in an inconsistent state (e.g., the database transaction commits, but the event publisher crashes before sending the message).
- **The Solution (Transactional Outbox Pattern):**
    1. Write both the primary business entity (e.g., Orders) and the outgoing event payload (e.g., Outbox) into the same database within a single local ACID transaction.
    2. Use an asynchronous polling worker or a **Change Data Capture (CDC)** engine (e.g., **Debezium**) to read the outbox table/transaction log (WAL) and publish messages to the message broker.
- **Change Data Capture (CDC) via Debezium:** Rather than relying on application code to poll the outbox table, CDC engines stream row-level changes directly from the database's write-ahead log (WAL) into the message broker with near-zero latency and minimal CPU overhead.

This guarantees **at-least-once publishing semantics**: an event is guaranteed to be emitted if and only if the local database transaction successfully commits.

Diagram: Transactional Outbox Pattern with CDC



4. Failure Recovery, DLQ, and Retries

Processing failures must be handled gracefully without dropping critical business data or deadlocking consumer pipelines.

> **The Challenge:** Unprocessable "poison pill" messages can cause infinite retry loops, exhausting system resources and blocking valid messages behind them.

Comprehensive Handling Strategy

- **Error Classification:** Distinguish between transient errors (e.g., temporary database connection timeout) and permanent errors (e.g., malformed payload or schema violation). Permanent errors must bypass retries and route directly to a Dead Letter Queue (DLQ).
- **Exponential Backoff & Jitter:** For transient errors, retry with exponentially increasing delays and randomized jitter to prevent thundering herd problems on downstream dependencies.
- **Dead Letter Queues (DLQ):** After reaching a maximum retry threshold (e.g., 3 attempts), automatically route the failing message to a designated DLQ partition.
- **Alerting & Re-injection:** Alert engineering teams immediately upon DLQ ingestion. Provide operator tooling to inspect, fix, and re-inject messages back into the primary workflow once resolved.

5. Schema Evolution & Governance

As systems evolve, event payloads inevitably change over time across decoupled engineering teams.

> **The Challenge:** Unannounced or non-backward-compatible producer changes can break downstream consumer applications at runtime.

- **Schema Registry:** Centralize and enforce explicit contracts using structured serialization protocols (e.g., Apache Avro, Protocol Buffers, or JSON Schema).
- **Compatibility Rules:** Enforce strict compatibility checks (Backward, Forward, or Full) at the registry level. Require producers to register new schemas before publishing; invalid schema mutations are rejected automatically at build or runtime.

6. Backpressure & Consumer Load Management

Consumers must remain resilient and stable when message production vastly exceeds downstream processing capacity.

> **The Challenge:** Sudden traffic spikes can overwhelm consumers, leading to Out-Of-Memory (OOM) crashes, unresponsiveness, or consumer group rebalancing storms (e.g., Kafka removing a consumer for failing to poll within timeout limits).

- **Rate Limiting & Reactive Streams:** Implement concurrency limits and control message fetching rates on the consumer client.
- **Decouple Fetching from Processing:** Use internal worker thread pools to process payloads asynchronously while maintaining a consistent heartbeat with the message broker.
- **Auto-scaling on Lag:** Trigger dynamic horizontal scaling of consumer instances based on real-time consumer lag metrics rather than CPU/Memory usage.

7. Monitoring & Observability

Debugging operational issues across asynchronous, decoupled microservices requires dedicated visibility patterns.

- **Consumer Lag:** The primary operational metric in Pub/Sub architectures. Measures the gap/delta between the latest published offset/message and the last processed offset by a consumer group.
- **Distributed Tracing (OpenTelemetry):** Inject trace contexts (e.g., traceId, spanId) into message headers at publish time. Consumers extract these headers to propagate execution contexts across service boundaries, enabling full end-to-end trace visualization.
- **Centralized Structured Logging:** Mandate structured JSON logging containing eventId, correlationId, topic, partition, consumerGroup, and explicit error stack traces for rapid querying and diagnostic analysis.

Team Best Practices & Guidelines

To maintain reliability, maintainability, and security across distributed event-driven systems, engineering teams must adopt standardized operational guidelines. Below are the implementation details for schema evolution, topic naming, and access security.

1. Schema Evolution & Governance

Events represent immutable public APIs between decoupled domains. Changes to message structures must be governed to ensure producers never break downstream consumers at runtime.

Implementation Patterns

- **Central Schema Registry:** Store and enforce message contracts in a centralized registry (e.g., Confluent Schema Registry, AWS Glue Schema Registry) using structured serialization formats like Apache Avro, Protocol Buffers, or JSON Schema.
- **Schema Versioning:** Embed schema IDs or version metadata into event headers (or payload metadata) so consumers can fetch and cache the correct deserializer dynamically.

Compatibility Strategies & Enforcement

Enforce strict compatibility checks at the registry level. Require producers to register new schemas before publishing; invalid schema mutations are rejected at build/deploy time.

- **Backward Compatibility (Default):** A new schema version can read data produced by previous schema versions. Downstream consumers can be upgraded **before** upgrading producers.
- **Forward Compatibility:** Previous schema versions can read data produced by the new schema version. Producers can be upgraded **before** consumers.
- **Full Compatibility:** Schema changes are both backward and forward compatible. Old and new consumers/producers can operate concurrently without deployment order constraints.

Golden Rules for Schema Compatibility

- Always provide default values for newly added fields.
- Never delete required fields; instead, mark them as deprecated (deprecated: true) and remove them only across major schema version migrations.

- Avoid changing field data types or renaming existing fields (prefer adding a new field alongside a deprecation plan).

2. Standardized Naming Conventions

A unified naming convention provides clear domain ownership, quick routing identification, and structured access control patterns across environments.

Recommended Topic Hierarchy Structure

Segment	Purpose	Examples
<environment>	Isolates traffic by lifecycle stage.	prod, staging, dev
<domain>	Identifies the top-level bounded context / service owner.	payments, logistics, identity
<subdomain> *(Optional)*	Further isolates sub-capabilities within large domains.	checkout, fulfillment
<entity>	The primary business object or aggregate root involved.	order, user, invoice
<event-type>	Describes the action in past tense or state change classification.	created, updated, payment-failed

Concrete Examples

- prod.payments.checkout.order.created
- staging.logistics.shipment.status-updated
- dev.identity.user.password-reset-requested

Additional Naming Rules

- **Casing:** Use lowercase kebab-case or snake_case consistently across all teams; avoid mixed casing (camelCase).
- **DLQ & Retry Extensions:** Standardize system queue extensions to match primary topics:
    - Retry Topic: prod.payments.checkout.order.created.retry

    - Dead Letter Queue: prod.payments.checkout.order.created.dlq

3. End-to-End Security & Access Control

Event streaming platforms handle sensitive business data and require strict isolation across network, transport, and application layers.

Data in Transit & Rest Encryption

- **Transport Layer Security (TLS):** Mandate mTLS (Mutual TLS) for all broker connections. This encrypts data in transit and enables strong cryptographic client authentication.
- **Encryption at Rest:** Ensure message logs on broker disks are encrypted using cloud-native KMS keys (e.g., AWS KMS, GCP KMS) or disk-level encryption (LUKS/dm-crypt).
- **Payload Encryption (Field-Level Encryption):** For high-sensitivity data (PII, PCI-DSS compliance), encrypt sensitive payload fields using client-side envelope encryption before publishing, storing keys in a dedicated vault (e.g., HashiCorp Vault).

Authentication & Access Control (IAM/ACLs)

- **Identity Federation:** Authenticate producers, consumers, and operators using enterprise identity providers via OAuth 2.0 / OIDC, SASL/SCRAM, or mTLS certificates.
- **Least Privilege ACLs:** Restrict access to topics, consumer groups, and schema registries using granular Access Control Lists (ACLs) or role-based access control (RBAC).
    - **Producers:** Granted write-only permissions (WRITE, DESCRIBE) restricted to specific domain topics (e.g., payments.*).
    - **Consumers:** Granted read-only permissions (READ, DESCRIBE) for specified topics, constrained to dedicated consumer-group names.
    - **Administrative Tools:** Disallow wildcard * permissions in production environments; limit admin rights to Automated Deployment / CI/CD pipelines.

Summary

Publish-Subscribe (Pub/Sub) is an asynchronous messaging pattern designed to facilitate communication between decoupled software systems. By using a central message broker, publishers can broadcast events without needing to know which subscribers will receive them, and subscribers can process events independently.**Core Philosophy & Benefits**

- **Decoupling:** Services operate in isolation, removing spatial, temporal, and implementation dependencies.
- **Scalability:** Supports horizontal scaling, allowing consumer pools to grow dynamically based on demand.
- **Resilience:** Acts as a buffer (load leveling) to protect downstream services during traffic spikes and ensures fault isolation.

Essential Building Blocks

- Publishers: Components that generate events; "fire and forget."
- Subscribers: Components that ingest events and execute business logic.
- Topics/Channels: Logical routing paths where messages are published.
- Message Broker: The infrastructure (e.g., Kafka, RabbitMQ) that manages routing and persistence.
- Payload: The data format (e.g., JSON, Protobuf) representing the event state.

Delivery & Routing Strategies

- **Delivery Models:**
  - *At-Most-Once:* High throughput, but messages may be lost.
  - *At-Least-Once:* Guaranteed delivery, but requires idempotent consumers to handle duplicates.
  - *Exactly-Once:* Ensures precise processing, though often at a performance cost.
- **Routing Patterns:** Includes Topic-based routing, Content-based filtering, and the Fan-Out pattern for parallel task execution.

Engineering Trade-offs & Challenges

- **Consistency:** Managing message ordering via partition keys and ensuring idempotency.
- **Reliability:** Utilizing Dead Letter Queues (DLQ) for failed messages and exponential backoff for retries.
- **Integration:** Solving the "Dual-Write" problem using the Transactional Outbox pattern.
- **Observability:** Focusing on key metrics like consumer lag and utilizing distributed tracing for end-to-end visibility.

Team Best Practices

- Schema Governance: Enforce strict contracts via centralized registries to prevent breaking changes.
- Standardization: Use consistent naming conventions (e.g., environment.domain.entity.event-type) to improve organization.
- Security: Implement mTLS, encryption at rest, and least-privilege access control (RBAC/ACLs) to protect sensitive data.

Gem

- GEM Link - https://gemini.google.com/gem/ec8b63517293

Reference material

- Google Cloud Pub/Sub Documentation
- Enterprise Integration Patterns: Publish-Subscribe Channel
- Apache Kafka Documentation
- RabbitMQ Documentation